

Graph Algorithms

A senior-interview study guide for internalizing graph fundamentals, reusable patterns, memorization targets, and the generalizations that connect the easy problems to the hard ones.

How To Use This Guide

This guide is designed for repeated passes. First, learn the mental model and the templates. Second, solve representative problems by pattern. Third, return to the invariants: what is true when a node is enqueued, when a node is marked visited, when a distance is finalized, and when a component is merged.

Memorize

Do not memorize entire solutions. Memorize the state, invariant, and loop shape. For graphs, the reusable unit is usually: build adjacency, initialize frontier, mark visited at the right time, process neighbors, update answer.

Study pass	Goal	What to drill
Pass 1	Recognize graph structure	Nodes, edges, directed vs undirected, weighted vs unweighted, grid as graph
Pass 2	Memorize skeletons	BFS, DFS, DSU, topo sort, Dijkstra
Pass 3	Attach invariants	Why visited works, why shortest path is valid, why cycle detection is correct
Pass 4	Senior level	Tradeoffs, edge cases, complexity, state-space modeling, explaining alternatives

Core Interview Promise

Most graph interviews are not about exotic algorithms. They test whether you can convert a messy problem into a graph, choose the right traversal or optimization rule, and maintain a clean invariant under pressure. Senior interviews add design judgment: when to precompute, when to avoid recursion, how to handle scale, and how to explain correctness.

1. Graph Basics

A graph is a set of nodes connected by edges. The first interview skill is not coding. It is deciding what the nodes are, what the edges mean, and whether edge direction or cost matters.

Property	Question to ask	Typical algorithm impact
Directed?	Can movement/dependency go one way only?	Topo sort, directed cycle detection, SCC
Weighted?	Do edges have costs?	Dijkstra, Bellman-Ford, MST, not plain BFS
Unweighted?	Is every step equal cost?	BFS gives shortest path by number of edges
Cyclic?	Can you return to a previous node?	Need visited states; cycle detection may be the task
Connected?	Can all nodes reach each other?	Count components with DFS/BFS/DSU
Dense?	Are many pairs connected?	Adjacency matrix may be viable; $O(V^2)$ scans may appear
Sparse?	Edges much fewer than V^2 ?	Adjacency list is standard
Memorize Adjacency list is the default LeetCode representation: dict/list from node to neighbors. For grids, the matrix itself is the graph and neighbors are generated on the fly.		

Representation Templates

```
# Undirected edge list to adjacency
adj = defaultdict(list)
for u, v in edges:
    adj[u].append(v)
    adj[v].append(u)

# Directed edge list to adjacency
adj = defaultdict(list)
for u, v in edges:
    adj[u].append(v)

# Weighted graph
adj = defaultdict(list)
for u, v, w in edges:
    adj[u].append((v, w))
```

Generalization

A graph does not need to look like a graph. Words connected by one-letter transformations, locks connected by digit turns, board states connected by legal moves, courses connected by prerequisites, accounts connected by shared emails, and islands connected by adjacent land are all graphs.

Problem surface	Node	Edge
Grid island	Cell	Up/down/left/right neighboring land
Word ladder	Word	One-character change
Course schedule	Course	Prerequisite relationship
Accounts merge	Email/account	Shared ownership
Open lock	Lock combination	One wheel turn
Network delay	Server	Directed weighted transmission

2. BFS and DFS

BFS and DFS are the two base traversals. BFS explores by distance layers. DFS explores deeply before backtracking. The interview move is to know what each traversal guarantees.

Traversal	Best for	Guarantee
BFS	Unweighted shortest path, levels, minimum steps	First time reaching a node is the shortest edge count
DFS	Components, reachability, backtracking, cycle shape	Explores an entire branch/component before returning
Iterative DFS	Avoid recursion limits	Same reachability behavior with explicit stack
Recursive DFS	Clean tree/component logic	Call stack stores the path implicitly
Memorize	BFS invariant: when a node is popped from the queue, it belongs to the current shortest-distance layer in an unweighted graph. DFS invariant: the active recursion path represents the current branch.	

BFS Skeleton

```
from collections import deque

def bfs(start):
    q = deque([start])
    seen = {start}
    steps = 0

    while q:
        for _ in range(len(q)):
            node = q.popleft()
            if is_goal(node):
                return steps
            for nei in neighbors(node):
                if nei not in seen:
                    seen.add(nei)          # mark when enqueued
                    q.append(nei)
        steps += 1
    return -1
```

DFS Skeleton

```
def dfs(node):
    if node in seen:
        return
    seen.add(node)
    for nei in adj[node]:
        dfs(nei)
```

Where People Slip

- Marking visited too late in BFS can enqueue the same node many times.
- Using DFS for shortest path in an unweighted graph usually creates extra complexity.
- For grid DFS, mutate the grid or maintain seen; do not accidentally revisit forever.
- For recursive DFS in Python, large graphs can exceed recursion depth; iterative DFS is safer for production-scale thinking.

Senior Interview Explanation

A senior answer should explicitly name the guarantee. For example: 'Because every edge has equal cost, BFS explores all nodes at distance k before distance $k+1$, so the first time we see the target is optimal.'
That sentence is often more important than another 10 lines of code.

3. Grids as Graphs

Grid problems are graph problems where nodes are coordinates and edges are local movements. You rarely build an adjacency list; you generate neighbors with direction arrays.

Memorize Grid neighbor loop: bounds check, obstacle/value check, seen check, then process. This one shape solves islands, rotting oranges, walls and gates, flood fill, shortest path in binary matrix, and many variants.

```
dirs = [(1, 0), (-1, 0), (0, 1), (0, -1)]
```

```
for dr, dc in dirs:
    nr, nc = r + dr, c + dc
    if 0 <= nr < rows and 0 <= nc < cols:
        if (nr, nc) not in seen and grid[nr][nc] != BLOCKED:
            seen.add((nr, nc))
            q.append((nr, nc))
```

Pattern	Use traversal	Key idea
Number of islands	DFS/BFS	Each new land cell starts a component
Max area of island	DFS/BFS	Return component size
Rotting oranges	Multi-source BFS	All initially rotten cells start at time 0
Walls and gates	Multi-source BFS	All gates start at distance 0
Pacific Atlantic water flow	Reverse DFS/BFS	Start from oceans and move uphill
Shortest path binary matrix	BFS	Eight-direction unweighted shortest path

Multi-Source BFS

Multi-source BFS is just BFS with many starting nodes. It is the right mental model when many sources spread simultaneously: fire, infection, gates, nearest zero, nearest exit, or minimum distance to any source.

```
q = deque()
seen = set()
for r in range(rows):
    for c in range(cols):
        if is_source(r, c):
            q.append((r, c))
            seen.add((r, c))

dist = 0
while q:
    for _ in range(len(q)):
        r, c = q.popleft()
        for nr, nc in valid_neighbors(r, c):
            if (nr, nc) not in seen:
                seen.add((nr, nc))
                q.append((nr, nc))
    dist += 1
```

Generalization: single-source BFS asks distance from one origin. Multi-source BFS asks distance from the nearest origin. Conceptually, you add a fake super-source connected to every real source with a zero-cost edge.

4. Components and Cycles

A connected component is a maximal set of nodes reachable from each other. Component problems ask you to count groups, size groups, merge groups, or determine whether everything belongs to one group.

```
components = 0
seen = set()

for node in nodes:
    if node not in seen:
        components += 1
        dfs(node)
```

Memorize

Outer loop over all nodes plus traversal from unseen nodes equals component counting. This is the heart of Number of Provinces, Connected Components, Islands, Accounts Merge, and many graph grouping tasks.

Cycle Detection

Graph type	Cycle method	Invariant
Undirected	DFS with parent	A seen neighbor that is not parent means a cycle
Directed	DFS colors	Re-entering a node on current path means a cycle
Directed prerequisites	Topo sort	If processed count < n, a cycle blocked progress
Incremental undirected edges	DSU	An edge connecting nodes already in same set creates a cycle

Directed DFS Colors

```
# 0 = unvisited, 1 = visiting/current path, 2 = done
color = [0] * n
```

```
def has_cycle(u):
    if color[u] == 1:
        return True
    if color[u] == 2:
        return False

    color[u] = 1
    for v in adj[u]:
        if has_cycle(v):
            return True
    color[u] = 2
    return False
```

The color model generalizes to recursion-path reasoning. 'Visiting' means this node is on the active call stack. Seeing it again means you found a back edge into the current path.

5. Topological Sort

Topological sort orders directed acyclic graphs so every prerequisite appears before the thing that depends on it. It is the natural pattern for courses, builds, task scheduling, package installs, alien dictionary, and dependency resolution.

Memorize Kahn's algorithm: compute indegrees, queue all zero-indegree nodes, pop one, reduce neighbors, enqueue new zero-indegree nodes. If output length is less than n , there is a cycle.

```
from collections import deque, defaultdict

adj = defaultdict(list)
indeg = [0] * n
for pre, course in edges:
    adj[pre].append(course)
    indeg[course] += 1

q = deque([i for i in range(n) if indeg[i] == 0])
order = []

while q:
    u = q.popleft()
    order.append(u)
    for v in adj[u]:
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)

valid = len(order) == n
```

DFS Topo Sort

```
order = []
state = [0] * n # 0 unvisited, 1 visiting, 2 done

def dfs(u):
    if state[u] == 1:
        return False
    if state[u] == 2:
        return True
    state[u] = 1
    for v in adj[u]:
        if not dfs(v):
            return False
    state[u] = 2
    order.append(u)
    return True

for i in range(n):
    if not dfs(i):
        return []
order.reverse()
```

Approach	When it feels best	Watch for
Kahn BFS	Need cycle detection and natural layer processing	Correct edge direction and indegree
DFS postorder	Need dependency expansion, recursive reasoning	Reverse final postorder
Priority queue Kahn	Need lexicographically smallest valid order	Heap instead of queue

6. Union-Find / DSU

Disjoint Set Union tracks which nodes belong to the same component while edges are added. It is the right tool when the problem is about merging groups, detecting redundant edges, or answering connectivity queries after unions.

Memorize DSU has two operations: find representative, union representatives. Path compression and union by rank/size make operations effectively constant time for interview purposes.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        return True
```

Problem cue	DSU interpretation
Are these connected?	Same representative
Merge accounts by email	Union emails or account ids sharing an email
Redundant connection	Union returns False when edge closes a cycle
Number of provinces	Union connected cities, count roots
Kruskal MST	Take cheapest edges that connect different components

DSU vs DFS/BFS

If the full graph is static and you just need components once, DFS/BFS is usually simpler. If edges are processed as merge events, or you need to detect whether each new edge connects two existing groups, DSU is usually cleaner.

7. Shortest Paths

Shortest path selection is mostly about edge weights. Plain BFS is for unweighted graphs. Dijkstra is for nonnegative weights. Bellman-Ford handles negative weights. Floyd-Warshall handles all-pairs shortest paths on smaller dense graphs.

Graph condition	Algorithm	Complexity shape
Unweighted	BFS	$O(V + E)$
Weights 0 or 1	0-1 BFS	$O(V + E)$ with deque
Nonnegative weights	Dijkstra	$O((V + E) \log V)$ with heap
Negative edges, no negative cycles	Bellman-Ford	$O(VE)$
All pairs, smaller n	Floyd-Warshall	$O(V^3)$
Memorize Dijkstra invariant: when a node is popped from the min-heap with its best known distance and is not stale, that distance is finalized, assuming all edge weights are nonnegative.		

Dijkstra Skeleton

```
import heapq

dist = [float("inf")] * n
dist[src] = 0
heap = [(0, src)]

while heap:
    d, u = heapq.heappop(heap)
    if d != dist[u]:
        continue
    for v, w in adj[u]:
        nd = d + w
        if nd < dist[v]:
            dist[v] = nd
            heapq.heappush(heap, (nd, v))
```

0-1 BFS

When weights are only 0 and 1, use a deque. A zero-cost move goes to the front; a one-cost move goes to the back. This preserves increasing distance order without a heap.

```
from collections import deque

dq = deque([src])
dist = [float("inf")] * n
dist[src] = 0

while dq:
    u = dq.popleft()
    for v, w in adj[u]: # w is 0 or 1
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            if w == 0:
                dq.appendleft(v)
            else:
                dq.append(v)
```

Generalization

BFS, 0-1 BFS, and Dijkstra are the same idea with different frontier data structures. Queue for equal edge costs, deque for 0/1 edge costs, heap for arbitrary nonnegative costs.

8. Minimum Spanning Tree

An MST connects all nodes with minimum total edge weight in an undirected weighted graph. It appears in connection-cost problems: connect cities, points, cables, roads, or networks.

Memorize

Kruskal: sort edges by weight, take an edge if DSU says it connects two different components. Stop after $n-1$ edges.

```
edges.sort(key=lambda x: x[2]) # (u, v, w)
dsu = DSU(n)
cost = 0
taken = 0
```

```
for u, v, w in edges:
    if dsu.union(u, v):
        cost += w
        taken += 1
        if taken == n - 1:
            break
```

```
possible = taken == n - 1
```

Algorithm	Mental model	Best when
Kruskal	Globally cheapest edges that do not form cycles	Edge list is natural
Prim	Grow one connected tree by cheapest frontier edge	Adjacency list is natural

MST differs from shortest path. MST minimizes total network build cost. Shortest path minimizes route cost from source to target. These are not interchangeable.

9. Advanced Graph Patterns

Bipartite Graphs

A graph is bipartite if nodes can be colored with two colors so no edge connects same-colored nodes. Use BFS/DFS coloring. This appears in possible bipartition, conflict grouping, and odd-cycle detection.

```
color = {}

for start in nodes:
    if start in color:
        continue
    color[start] = 0
    q = deque([start])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in color:
                color[v] = 1 - color[u]
                q.append(v)
            elif color[v] == color[u]:
                return False
    return True
```

Strongly Connected Components

In a directed graph, an SCC is a group where every node can reach every other node. Senior interviews rarely require Tarjan from scratch, but knowing the idea helps: SCCs compress a directed graph into a DAG.

Good to know

Tarjan's algorithm uses discovery times and low-link values. Memorize the concept before the code: low-link tells you the earliest reachable ancestor from a DFS subtree.

Articulation Points and Bridges

A bridge is an edge whose removal disconnects the graph. An articulation point is a node whose removal disconnects the graph. These use DFS low-link reasoning. The common LeetCode hard version is Critical Connections in a Network.

Eulerian Paths

Eulerian path problems ask for a route that uses every edge exactly once. Reconstruct Itinerary is the classic version: use all tickets once, choose lexical order, and apply Hierholzer's algorithm.

```
def visit(u):
    heap = adj[u]
    while heap:
        v = heappop(heap)
        visit(v)
    route.append(u)

visit("JFK")
route.reverse()
```

State-Space Graphs

In state-space problems, each node is not a simple input item but a full state: position plus keys, remaining eliminations, bitmask of visited houses, or current word. The biggest senior-level mistake is using too little visited state.

Memorize

Visited must include every variable that changes future possibilities. If position is same but keys, remaining budget, or mask differ, those are different graph states.

Problem style	State should include
Shortest path with obstacle eliminations	(row, col, remaining_eliminations)
Collect all keys	(row, col, key_mask)
Traveling all nodes	(node, visited_mask)
Word ladder	current word
Open lock	current combination

10. Senior Interview Playbook

Recognition Checklist

- Can I model the input as nodes and edges?
- Are edges weighted or unweighted?
- Do I need shortest path, reachability, ordering, grouping, or minimum connection cost?
- Is the graph explicit, or do I generate neighbors dynamically?
- Does visited need to include extra state beyond node identity?
- Can I prove the first answer found is optimal?

Algorithm Selection Map

Need	Default pick	Why
Reachability/component size	DFS/BFS	Traverse all reachable nodes
Minimum steps unweighted	BFS	Layer order equals distance
Minimum cost nonnegative	Dijkstra	Heap expands by best known cost
Dependency order	Topological sort	DAG ordering
Merge dynamic groups	DSU	Efficient connectivity under unions
Minimum network cost	MST	Connect all nodes cheaply
Two-color conflict split	Bipartite BFS/DFS	Alternate colors across edges
Use every edge once	Eulerian path	Edge exhaustion route

How to Speak Correctness

A strong correctness explanation has three parts: invariant, progress, termination. Example for BFS: the queue processes nodes in nondecreasing distance; each unseen neighbor is assigned the first distance at which it can be reached; the search terminates when the target is found or the finite graph is exhausted.

Complexity Language

Use V for vertices and E for edges. For grids, $V = \text{rows} * \text{cols}$ and E is proportional to V because each cell has constant neighbors. For state-space BFS, V is the number of reachable states, which may be position times masks or budget values.

Algorithm	Time	Space
BFS/DFS adjacency list	$O(V + E)$	$O(V)$
Grid BFS/DFS	$O(RC)$	$O(RC)$
Topological sort	$O(V + E)$	$O(V + E)$ for adjacency plus indegree
DSU	$O(E \alpha(V))$	$O(V)$
Dijkstra heap	$O((V + E) \log V)$	$O(V + E)$
Kruskal MST	$O(E \log E)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$

Problem Drill Set

Stage	Problems to practice	Target internalization
Basics	Number of Islands, Flood Fill, Max Area of Island	Grid traversal and components
BFS	Rotting Oranges, Walls and Gates, Word Ladder, Open Lock	Layered minimum steps
DFS cycles	Course Schedule, Reconstruct Itinerary	Recursion state and postorder
Topo	Course Schedule II, Alien Dictionary	Indegree and dependency order
DSU	Number of Provinces, Accounts Merge, Redundant Connection	Merge semantics
Shortest path	Network Delay Time, Cheapest Flights Within K Stops, Path With Minimum Effort	Pick BFS/Dijkstra/stateful search correctly
Advanced	Critical Connections, Shortest Path to Get All Keys	Low-link and state-space visited

Final Mental Compression

Graph algorithms compress into five verbs: traverse, group, order, optimize, and model state. BFS/DFS traverse. DSU groups. Topological sort orders. Dijkstra/MST optimize under different objectives. State-space modeling turns a problem that does not look like a graph into one where those tools apply.

11. Memorization Sheet

Pattern	Memorize this
BFS	Queue, seen at enqueue time, level loop for steps
DFS	Seen guard, process node, recurse neighbors
Components	Outer loop over all nodes; traverse unseen; count++
Grid	Direction array plus bounds/value/seen checks
Multi-source BFS	Seed queue with all sources at time 0
Directed cycle	0/1/2 colors; seeing 1 means cycle
Topo Kahn	Indegree, zero queue, reduce neighbors, output count
DSU	find with path compression; union by size/rank; union false means already connected
Dijkstra	dist array, min-heap, stale skip, relax edges
MST Kruskal	Sort edges, union if different, take n-1 edges
State BFS	Visited key must include full future-relevant state

What Not To Memorize Blindly

- Exact variable names or full LeetCode solutions.
- Tarjan code before understanding low-link.
- Dijkstra for unweighted graphs when BFS is simpler.
- A single visited set when the problem has extra state such as keys, stops, or remaining budget.
- Recursive DFS in contexts where input size obviously risks stack overflow.

One-Sentence Invariants

- BFS: nodes are processed in increasing number of edges from the source.
- DFS component: after DFS returns, every reachable unseen node from the start has been marked.
- Topo sort: a node is emitted only after all incoming dependencies have been removed.
- DSU: two nodes are connected exactly when their representatives match.
- Dijkstra: the next non-stale heap pop has the smallest final distance among unsettled nodes.
- Kruskal: every accepted edge connects two previously separate components and never creates a cycle.